

ShopNest DevOps Project

Complete End-to-End Hands-On Guide

PHASE 2 — LOCAL DEVELOPMENT SETUP

What you will build in this phase

React Frontend | Node.js REST API | PostgreSQL Database | Redis Cache

All running together on your laptop — no cloud needed yet

PHASE 2 OVERVIEW

In Phase 1 you designed the architecture. In Phase 2 you build the actual application. By the end of this phase a real ShopNest application will be running on your laptop — frontend, backend, database, and cache all talking to each other.

i Phase 2 Goal

By the end of this phase you will have:

A React frontend running at `http://localhost:3000`

A Node.js REST API running at `http://localhost:5000`

PostgreSQL database running at `localhost:5432`

Redis cache running at `localhost:6379`

All 4 services talking to each other and validated with health checks

Every API endpoint tested and documented

PKG Phase 2 Deliverables

frontend/ — complete React application source code

backend/ — complete Node.js API source code

database/ — PostgreSQL schema and seed data

.env files — all environment variables defined

Working health check at `GET /health` on the API

Postman collection with all API tests passing

Step	What You Do	Time Estimate
1	Install PostgreSQL + Redis locally	10 min
2	Set up the Node.js backend project	15 min
3	Create database schema and migrations	15 min
4	Write all backend API routes and controllers	45 min
5	Set up environment variables (.env)	5 min
6	Set up the React frontend project	15 min
7	Write frontend pages and connect to the API	45 min
8	Run all 4 services and verify they connect	10 min
9	Test every API endpoint with curl	20 min
10	Add health checks and validate Phase 2 is complete	10 min

STEP 1 — INSTALL POSTGRESQL & REDIS LOCALLY

We need PostgreSQL and Redis running on your laptop for local development. In production these will be managed AWS services (RDS and ElastiCache), but locally we run them directly so development is fast and free.

1.1 — Install PostgreSQL

STEP 1 Install PostgreSQL 15

```
# Add the PostgreSQL APT repository
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-
pgdg main" > /etc/apt/sources.list.d/pgdg.list'

wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo apt-
key add -

# Update and install PostgreSQL 15
sudo apt-get update
sudo apt-get install -y postgresql-15 postgresql-client-15

# Start the PostgreSQL service
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Verify it is running
sudo systemctl status postgresql
# Look for: Active: active (running)
```

STEP 2 Create the ShopNest database and user

Switch to the postgres system user and open the PostgreSQL shell:

```
sudo -u postgres psql
```

You are now inside the PostgreSQL prompt. Type each command below and press Enter:

```
-- Create a dedicated database user
CREATE USER shopnest_user WITH PASSWORD 'shopnest_pass_2024';

-- Create the database
CREATE DATABASE shopnest_db OWNER shopnest_user;

-- Grant all privileges
GRANT ALL PRIVILEGES ON DATABASE shopnest_db TO shopnest_user;

-- Exit the PostgreSQL shell
\q
```

Test the connection with your new user:

```
psql -U shopnest_user -h localhost -d shopnest_db -W
# When prompted, type: shopnest_pass_2024
# You should see: shopnest_db=>
# Type \q to exit
```

OK Success Check

If you see the `shopnest_db=>` prompt, PostgreSQL is working correctly.
If you get 'connection refused', run: `sudo systemctl start postgresql`

1.2 — Install Redis ---

STEP 3 Install and start Redis

```
# Install Redis
sudo apt-get install -y redis-server

# Edit Redis config to allow connections from localhost only
sudo sed -i 's/^# bind 127.0.0.1/bind 127.0.0.1/' /etc/redis/redis.conf

# Start Redis
sudo systemctl start redis-server
sudo systemctl enable redis-server

# Verify Redis is running
redis-cli ping
# Expected output: PONG
```

i Local Development Port Summary

After Step 1, these services are running on your machine:

PostgreSQL -> localhost:5432 (database)
Redis -> localhost:6379 (cache)

We will add Node.js (port 5000) and React (port 3000) in Steps 2 and 6.

STEP 2 — SET UP THE NODE.JS BACKEND PROJECT

The backend is the heart of the application. It handles all business logic, validates requests, manages authentication, and communicates with both PostgreSQL and Redis.

2.1 — Initialise the Backend Project

STEP 4 Create the Node.js project

```
# Make sure you are in the project root
cd ~/shopnest

# Enter the backend folder
cd backend

# Initialise npm project (accepts all defaults)
npm init -y

# Install all required dependencies
npm install express pg redis bcryptjs jsonwebtoken cors helmet morgan dotenv
express-validator

# Install development dependencies
npm install --save-dev nodemon jest supertest
```

i What Each Package Does

express	— Web framework for creating API routes
pg	— PostgreSQL client for Node.js
redis	— Redis client for Node.js
bcryptjs	— Hash passwords before storing them
jsonwebtoken	— Create and verify JWT auth tokens
cors	— Allow the React frontend to call the API
helmet	— Set security HTTP headers automatically
morgan	— Log every incoming HTTP request
dotenv	— Load environment variables from .env file
express-validator	— Validate request body inputs
nodemon	— Auto-restart the server when code changes (dev only)
jest + supertest	— Run automated API tests

2.2 — Create the Backend File Structure

STEP 5 Create all source files

```
# Make sure you are inside backend/
```

```
cd ~/shopnest/backend

# Create all the files we will fill in
touch src/index.js
touch src/controllers/authController.js
touch src/controllers/productController.js
touch src/controllers/cartController.js
touch src/controllers/orderController.js
touch src/routes/authRoutes.js
touch src/routes/productRoutes.js
touch src/routes/cartRoutes.js
touch src/routes/orderRoutes.js
touch src/middleware/auth.js
touch src/middleware/errorHandler.js
touch src/models/db.js
touch src/models/redis.js
touch .env
touch .env.example
```

Verify the structure:

```
find . -type f | sort
# You should see all the files listed above
```

2.3 — Create the Environment File

STEP 6 Write the .env file

The .env file holds all secrets and config. It is NEVER committed to Git. Open the file in your text editor:

```
nano ~/shopnest/backend/.env
```

Type the following content exactly, then press Ctrl+O to save and Ctrl+X to exit:

```
# — Server —
NODE_ENV=development
PORT=5000

# — Database —
DB_HOST=localhost
DB_PORT=5432
DB_NAME=shopnest_db
DB_USER=shopnest_user
DB_PASSWORD=shopnest_pass_2024

# — Redis —
REDIS_HOST=localhost
REDIS_PORT=6379

# — JWT Auth —
JWT_SECRET=shopnest_super_secret_jwt_key_change_in_production
JWT_EXPIRES_IN=24h
```

```
# — CORS —  
FRONTEND_URL=http://localhost:3000
```

! Important Security Rule

Never commit this .env file to Git. The .gitignore from Phase 1 already blocks it.
In production (Phases 5-8) these values will come from Jenkins credentials, AWS Secrets Manager, or Kubernetes secrets — never from a .env file.

Create a .env.example file that IS committed (no real secrets — just keys with placeholder values):

```
cat > ~/shopnest/backend/.env.example << 'EOF'  
NODE_ENV=development  
PORT=5000  
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=shopnest_db  
DB_USER=your_db_user  
DB_PASSWORD=your_db_password  
REDIS_HOST=localhost  
REDIS_PORT=6379  
JWT_SECRET=your_jwt_secret_here  
JWT_EXPIRES_IN=24h  
FRONTEND_URL=http://localhost:3000  
EOF
```

2.4 — Write the Database Connection

STEP 7 Create src/models/db.js

Open the file:

```
nano ~/shopnest/backend/src/models/db.js
```

Type the following code:

```
const { Pool } = require('pg');  
require('dotenv').config();  
  
// Create a connection pool  
// A pool keeps several database connections open and reuses them  
// This is much faster than opening a new connection on every request  
const pool = new Pool({  
  host: process.env.DB_HOST,  
  port: process.env.DB_PORT,  
  database: process.env.DB_NAME,  
  user: process.env.DB_USER,  
  password: process.env.DB_PASSWORD,  
  max: 10, // maximum 10 connections in the pool  
  idleTimeoutMillis: 30000,  
  connectionTimeoutMillis: 2000,  
});
```

```
});

// Test the connection when the server starts
pool.connect((err, client, release) => {
  if (err) {
    console.error('ERROR connecting to PostgreSQL:', err.message);
    return;
  }
  console.log('PostgreSQL connected successfully');
  release();
});

module.exports = pool;
```

2.5 — Write the Redis Connection

STEP 8 Create src/models/redis.js

```
nano ~/shopnest/backend/src/models/redis.js
```

```
const redis = require('redis');
require('dotenv').config();

const redisClient = redis.createClient({
  socket: {
    host: process.env.REDIS_HOST,
    port: process.env.REDIS_PORT,
  }
});

redisClient.on('error', (err) => console.error('Redis error:', err));
redisClient.on('connect', () => console.log('Redis connected successfully'));

// Connect to Redis when this module loads
(async () => { await redisClient.connect(); })();

module.exports = redisClient;
```

2.6 — Write the Authentication Middleware

STEP 9 Create src/middleware/auth.js

This middleware runs BEFORE any protected route handler. It checks the JWT token in the request header.

```
nano ~/shopnest/backend/src/middleware/auth.js
```

```
const jwt = require('jsonwebtoken');
```

```
const auth = (req, res, next) => {
  // Get the token from the Authorization header
  // Header format: Authorization: Bearer <token>
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    // Verify the token using our JWT_SECRET
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach user data to the request
    next(); // pass control to the next handler
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
};

module.exports = auth;
```

2.7 — Write the Error Handler Middleware

STEP 10 Create src/middleware/errorHandler.js

```
nano ~/shopnest/backend/src/middleware/errorHandler.js
```

```
// Global error handler - catches any error thrown in a route handler
// Must have 4 parameters (err, req, res, next) - Express identifies it by this
const errorHandler = (err, req, res, next) => {
  console.error('ERROR:', err.stack);

  // If status code was set on the error, use it - otherwise default to 500
  const statusCode = err.statusCode || 500;
  const message = err.message || 'Internal Server Error';

  res.status(statusCode).json({
    error: message,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack }),
  });
};

module.exports = errorHandler;
```

STEP 3 — CREATE DATABASE SCHEMA & MIGRATIONS

A migration is a versioned SQL file that changes the database schema. Using migration files means you can recreate the entire database from scratch, share schema changes with your team, and roll back if something goes wrong.

3.1 — Create the Migration Files

STEP 11 Create migration 001 — users table

```
nano ~/shopnest/database/migrations/001_create_users.sql
```

```
-- Migration 001: Create users table
-- Run this FIRST — other tables reference users

CREATE TABLE IF NOT EXISTS users (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,          -- bcrypt hash, never plain text
  first_name  VARCHAR(100) NOT NULL,
  last_name   VARCHAR(100) NOT NULL,
  role        VARCHAR(20)  NOT NULL DEFAULT 'customer', -- customer or admin
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Index on email for fast login lookups
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);

SELECT 'users table created' AS status;
```

STEP 12 Create migration 002 — products table

```
nano ~/shopnest/database/migrations/002_create_products.sql
```

```
-- Migration 002: Create products table

CREATE TABLE IF NOT EXISTS products (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  description TEXT,
  price       DECIMAL(10,2) NOT NULL,
  stock       INTEGER NOT NULL DEFAULT 0,
  category    VARCHAR(100),
  image_url   VARCHAR(500),
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```
);

-- Index on category for filtering
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category);

SELECT 'products table created' AS status;
```

STEP 13 Create migration 003 — orders and order_items tables

```
nano ~/shopnest/database/migrations/003_create_orders.sql
```

```
-- Migration 003: Create orders and order_items tables

CREATE TABLE IF NOT EXISTS orders (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',
  total_amount DECIMAL(10,2) NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS order_items (
  id          SERIAL PRIMARY KEY,
  order_id    INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  INTEGER NOT NULL REFERENCES products(id),
  quantity    INTEGER NOT NULL,
  unit_price  DECIMAL(10,2) NOT NULL,
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_orders_user      ON orders(user_id);
CREATE INDEX IF NOT EXISTS idx_order_items_order ON order_items(order_id);

SELECT 'orders tables created' AS status;
```

STEP 14 Run all migrations

Run all migration files against your local database:

```
# Run all migration files in order
psql -U shopnest_user -h localhost -d shopnest_db -W \
  -f ~/shopnest/database/migrations/001_create_users.sql

psql -U shopnest_user -h localhost -d shopnest_db -W \
  -f ~/shopnest/database/migrations/002_create_products.sql

psql -U shopnest_user -h localhost -d shopnest_db -W \
  -f ~/shopnest/database/migrations/003_create_orders.sql

# Verify tables were created
psql -U shopnest_user -h localhost -d shopnest_db -W -c '\dt'
# Expected output:
```

```
# Schema |      Name      | Type |      Owner
# public | order_items | table | shopnest_user
# public | orders      | table | shopnest_user
# public | products    | table | shopnest_user
# public | users       | table | shopnest_user
```

STEP 15 Create seed data — insert sample products

```
nano ~/shopnest/database/seeds/001_seed_products.sql
```

```
-- Seed data: 6 sample products to test with

INSERT INTO products (name, description, price, stock, category) VALUES
  ('Wireless Headphones', 'Noise cancelling Bluetooth headphones', 89.99, 50,
  'Electronics'),
  ('Running Shoes',      'Lightweight marathon running shoes',    129.99, 30,
  'Sports'),
  ('Coffee Maker',       '12-cup programmable coffee maker',     59.99, 25,
  'Kitchen'),
  ('Yoga Mat',           'Non-slip eco-friendly yoga mat',    34.99, 100,
  'Sports'),
  ('Desk Lamp',          'LED adjustable brightness desk lamp', 44.99, 75,
  'Home'),
  ('Backpack',           'Water-resistant 30L hiking backpack', 79.99, 40,
  'Sports');

SELECT 'seed data inserted' AS status;
SELECT COUNT(*) AS product_count FROM products;
```

Run the seed file:

```
psql -U shopnest_user -h localhost -d shopnest_db -W \
-f ~/shopnest/database/seeds/001_seed_products.sql
```

```
# Expected output:
#      status
# -----
# seed data inserted
#
# product_count
# -----
#           6
```

STEP 4 — WRITE THE API ROUTES & CONTROLLERS

The API is structured in two layers: Routes define the URL pattern and HTTP method. Controllers contain the actual logic. This separation keeps the code clean and testable.

HTTP Method	Route	What it does	Auth Required?
POST	/api/auth/register	Create a new user account	No
POST	/api/auth/login	Log in — returns JWT token	No
GET	/api/products	List all products (with filters)	No
GET	/api/products/:id	Get single product details	No
POST	/api/products	Create a product (admin only)	Yes — admin
GET	/api/cart	Get current user cart from Redis	Yes
POST	/api/cart	Add item to cart	Yes
DELETE	/api/cart/:productId	Remove item from cart	Yes
POST	/api/orders	Place an order from cart contents	Yes
GET	/api/orders	List current user orders	Yes
GET	/api/orders/:id	Get single order details	Yes
GET	/health	Health check — used by load balancer	No

4.1 — Write the Auth Controller

STEP 16 Create src/controllers/authController.js

```
nano ~/shopnest/backend/src/controllers/authController.js
```

```
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');
const pool   = require('../models/db');

// POST /api/auth/register
const register = async (req, res, next) => {
  try {
    const { email, password, first_name, last_name } = req.body;

    // Check if user already exists
    const existing = await pool.query('SELECT id FROM users WHERE email = $1',
[email]);
    if (existing.rows.length > 0) {
      return res.status(409).json({ error: 'Email already registered' });
    }
  }
}
```

```

    // Hash the password (never store plain text)
    const hashedPassword = await bcrypt.hash(password, 12);

    // Insert the new user
    const result = await pool.query(
      'INSERT INTO users (email, password, first_name, last_name) VALUES ($1,$2,$3,$4) RETURNING id, email, first_name, last_name, role',
      [email, hashedPassword, first_name, last_name]
    );

    const user = result.rows[0];
    const token = jwt.sign({ userId: user.id, role: user.role },
      process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN });

    res.status(201).json({ token, user });
  } catch (err) { next(err); }
};

// POST /api/auth/login
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const result = await pool.query('SELECT * FROM users WHERE email = $1',
      [email]);
    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const user = result.rows[0];
    const match = await bcrypt.compare(password, user.password);
    if (!match) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const token = jwt.sign({ userId: user.id, role: user.role },
      process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN });

    res.json({ token, user: { id: user.id, email: user.email, first_name:
      user.first_name, role: user.role } });
  } catch (err) { next(err); }
};

module.exports = { register, login };

```

4.2 — Write the Product Controller

STEP 17 Create src/controllers/productController.js

```
nano ~/shopnest/backend/src/controllers/productController.js
```

```
const pool = require('../models/db');
```

```
// GET /api/products (supports ?category=Sports&limit=10&offset=0)
const getAllProducts = async (req, res, next) => {
  try {
    const { category, limit = 20, offset = 0 } = req.query;
    let query = 'SELECT * FROM products';
    const params = [];

    if (category) {
      query += ' WHERE category = $1';
      params.push(category);
    }

    query += ` ORDER BY created_at DESC LIMIT ${params.length + 1} OFFSET
    ${params.length + 2}`;
    params.push(limit, offset);

    const result = await pool.query(query, params);
    res.json({ products: result.rows, count: result.rowCount });
  } catch (err) { next(err); }
};

// GET /api/products/:id
const getProductById = async (req, res, next) => {
  try {
    const result = await pool.query('SELECT * FROM products WHERE id = $1',
    [req.params.id]);
    if (result.rows.length === 0) return res.status(404).json({ error: 'Product
    not found' });
    res.json({ product: result.rows[0] });
  } catch (err) { next(err); }
};

// POST /api/products (admin only)
const createProduct = async (req, res, next) => {
  try {
    if (req.user.role !== 'admin') return res.status(403).json({ error: 'Admin
    access required' });
    const { name, description, price, stock, category, image_url } = req.body;
    const result = await pool.query(
      'INSERT INTO products (name, description, price, stock, category,
    image_url) VALUES ($1,$2,$3,$4,$5,$6) RETURNING *',
      [name, description, price, stock, category, image_url]
    );
    res.status(201).json({ product: result.rows[0] });
  } catch (err) { next(err); }
};

module.exports = { getAllProducts, getProductById, createProduct };
```

4.3 — Write the Cart Controller

STEP 18 Create src/controllers/cartController.js

The cart is stored in Redis, not PostgreSQL. This is intentional — carts change constantly as users browse, and Redis handles these fast, temporary writes much better than a relational database.

```
nano ~/shopnest/backend/src/controllers/cartController.js
```

```

const redisClient = require('../models/redis');
const pool        = require('../models/db');

// GET /api/cart
const getCart = async (req, res, next) => {
  try {
    const cartKey = `cart:${req.user.userId}`;
    const cartData = await redisClient.get(cartKey);
    const cart     = cartData ? JSON.parse(cartData) : [];
    res.json({ cart });
  } catch (err) { next(err); }
};

// POST /api/cart
const addToCart = async (req, res, next) => {
  try {
    const { productId, quantity } = req.body;
    const cartKey = `cart:${req.user.userId}`;

    // Verify product exists in DB
    const product = await pool.query('SELECT id, name, price FROM products WHERE id = $1', [productId]);
    if (product.rows.length === 0) return res.status(404).json({ error: 'Product not found' });

    // Get existing cart or start empty
    const cartData = await redisClient.get(cartKey);
    const cart = cartData ? JSON.parse(cartData) : [];

    // Check if item already in cart
    const existingIndex = cart.findIndex(item => item.productId === productId);
    if (existingIndex >= 0) {
      cart[existingIndex].quantity += quantity;
    } else {
      cart.push({ productId, quantity, name: product.rows[0].name, price: product.rows[0].price });
    }

    // Save cart to Redis with 24-hour expiry
    await redisClient.setEx(cartKey, 86400, JSON.stringify(cart));
    res.json({ cart, message: 'Item added to cart' });
  } catch (err) { next(err); }
};

// DELETE /api/cart/:productId
const removeFromCart = async (req, res, next) => {
  try {
    const cartKey = `cart:${req.user.userId}`;
    const cartData = await redisClient.get(cartKey);
    if (!cartData) return res.json({ cart: [] });

    const cart = JSON.parse(cartData).filter(item => item.productId !== parseInt(req.params.productId));
    await redisClient.setEx(cartKey, 86400, JSON.stringify(cart));
    res.json({ cart, message: 'Item removed from cart' });
  } catch (err) { next(err); }
};

module.exports = { getCart, addToCart, removeFromCart };

```

4.4 — Write the Order Controller

STEP 19 Create src/controllers/orderController.js

```
nano ~/shopnest/backend/src/controllers/orderController.js
```

```
const pool      = require('../models/db');
const redisClient = require('../models/redis');

// POST /api/orders - convert cart into a real order
const placeOrder = async (req, res, next) => {
  const client = await pool.connect();
  try {
    const cartKey = `cart:${req.user.userId}`;
    const cartData = await redisClient.get(cartKey);
    if (!cartData || JSON.parse(cartData).length === 0) {
      return res.status(400).json({ error: 'Cart is empty' });
    }

    const cart      = JSON.parse(cartData);
    const totalAmount = cart.reduce((sum, item) => sum + item.price *
item.quantity, 0);

    // Use a database transaction - either all inserts succeed or none do
    await client.query('BEGIN');

    const orderResult = await client.query(
      'INSERT INTO orders (user_id, total_amount, status) VALUES ($1,$2,$3)
RETURNING *',
      [req.user.userId, totalAmount.toFixed(2), 'pending']
    );
    const order = orderResult.rows[0];

    // Insert each cart item as an order_item row
    for (const item of cart) {
      await client.query(
        'INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES ($1,$2,$3,$4)',
        [order.id, item.productId, item.quantity, item.price]
      );
    }

    await client.query('COMMIT');

    // Clear the cart from Redis
    await redisClient.del(cartKey);

    res.status(201).json({ order, message: 'Order placed successfully' });
  } catch (err) {
    await client.query('ROLLBACK');
    next(err);
  } finally {
    client.release();
  }
};

// GET /api/orders
const getOrders = async (req, res, next) => {
  try {
```

```

    const result = await pool.query(
      'SELECT o.*, json_agg(oi) AS items FROM orders o LEFT JOIN order_items oi
      ON o.id = oi.order_id WHERE o.user_id = $1 GROUP BY o.id ORDER BY o.created_at
      DESC',
      [req.user.userId]
    );
    res.json({ orders: result.rows });
  } catch (err) { next(err); }
};

module.exports = { placeOrder, getOrders };

```

4.5 — Write the Route Files

STEP 20 Create all 4 route files

Create `src/routes/authRoutes.js`:

```
nano ~/shopnest/backend/src/routes/authRoutes.js
```

```

const express = require('express');
const { register, login } = require('../controllers/authController');
const router = express.Router();

router.post('/register', register);
router.post('/login', login);

module.exports = router;

```

Create `src/routes/productRoutes.js`:

```
nano ~/shopnest/backend/src/routes/productRoutes.js
```

```

const express = require('express');
const { getAllProducts, getProductById, createProduct } =
  require('../controllers/productController');
const auth = require('../middleware/auth');
const router = express.Router();

router.get('/', getAllProducts);
router.get('/:id', getProductById);
router.post('/', auth, createProduct); // auth required

module.exports = router;

```

Create `src/routes/cartRoutes.js`:

```
nano ~/shopnest/backend/src/routes/cartRoutes.js
```

```
const express = require('express');
const { getCart, addToCart, removeFromCart } =
  require('../controllers/cartController');
const auth = require('../middleware/auth');
const router = express.Router();

router.get('/', auth, getCart);
router.post('/', auth, addToCart);
router.delete('/:productId', auth, removeFromCart);

module.exports = router;
```

Create `src/routes/orderRoutes.js`:

```
nano ~/shopnest/backend/src/routes/orderRoutes.js
```

```
const express = require('express');
const { placeOrder, getOrders } = require('../controllers/orderController');
const auth = require('../middleware/auth');
const router = express.Router();

router.post('/', auth, placeOrder);
router.get('/', auth, getOrders);

module.exports = router;
```

4.6 — Write the Main Entry Point

STEP 21 Create `src/index.js` — the main server file

```
nano ~/shopnest/backend/src/index.js
```

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const errorHandler = require('../middleware/errorHandler');

// Import all route files
const authRoutes = require('../routes/authRoutes');
const productRoutes = require('../routes/productRoutes');
const cartRoutes = require('../routes/cartRoutes');
const orderRoutes = require('../routes/orderRoutes');

const app = express();

// — Security & Utility Middleware —
app.use(helmet()); // Set security headers (XSS, clickjacking, etc.)
app.use(cors({ // Allow frontend to call the API
  origin: process.env.FRONTEND_URL,
```

```
    credentials: true,
  });
  app.use(morgan('dev')); // Log: GET /api/products 200 42ms
  app.use(express.json()); // Parse JSON request bodies

  // — Health Check —
  // Used by ALB, Kubernetes liveness probe, and monitoring
  app.get('/health', (req, res) => {
    res.json({
      status: 'healthy',
      timestamp: new Date().toISOString(),
      uptime: process.uptime(),
      version: process.env.npm_package_version || '1.0.0',
    });
  });

  // — API Routes —
  app.use('/api/auth', authRoutes);
  app.use('/api/products', productRoutes);
  app.use('/api/cart', cartRoutes);
  app.use('/api/orders', orderRoutes);

  // — 404 Handler —
  app.use((req, res) => {
    res.status(404).json({ error: `Route ${req.method} ${req.path} not found` });
  });

  // — Global Error Handler —
  app.use(errorHandler);

  // — Start Server —
  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`ShopNest API running on http://localhost:${PORT}`);
    console.log(`Health check: http://localhost:${PORT}/health`);
    console.log(`Environment: ${process.env.NODE_ENV}`);
  });
```

STEP 22 Add npm scripts to package.json

```
nano ~/shopnest/backend/package.json
```

Find the "scripts" section and replace it with:

```
"scripts": {
  "start": "node src/index.js",
  "dev": "nodemon src/index.js",
  "test": "jest --coverage",
  "test:watch": "jest --watch"
},
```

STEP 23 Start the backend server and verify

```
cd ~/shopnest/backend
```

```
# Start in development mode (nodemon restarts on file changes)
npm run dev
```

You should see output like this in the terminal:

```
[nodemon] starting `node src/index.js`
PostgreSQL connected successfully
Redis connected successfully
ShopNest API running on http://localhost:5000
Health check: http://localhost:5000/health
Environment: development
```

OK Success Check — open a second terminal and run:

```
curl http://localhost:5000/health
```

Expected response:

```
{ "status": "healthy", "timestamp": "2024-...", "uptime": 2.1, "version": "1.0.0" }
```

STEP 5 — SET UP THE REACT FRONTEND

The frontend is a React single-page application. It makes API calls to the Node.js backend and renders the UI. Nginx will serve the built React files and proxy API calls to the backend.

5.1 — Initialise the React Project

STEP 24 Create the React application

Open a NEW terminal window. Keep the backend terminal running.

```
# Go to the project root (not inside backend/)
cd ~/shopnest

# Create React app inside the existing frontend folder
npx create-react-app frontend --template cra-template
# This takes 2-3 minutes - it downloads many packages

cd frontend

# Install additional packages for the frontend
npm install axios react-router-dom

# axios          - makes HTTP calls to our backend API
# react-router-dom - handles navigation between pages
```

5.2 — Create the Frontend Environment File

STEP 25 Create frontend/.env

```
# The React app needs to know where the API is
# REACT_APP_ prefix is required - Create React App only exposes variables with
this prefix

echo "REACT_APP_API_URL=http://localhost:5000/api" > ~/shopnest/frontend/.env
```

5.3 — Create the API Service Layer

STEP 26 Create src/services/api.js — centralised API calls

Instead of writing fetch/axios calls in every component, we create a single service file. If the API URL changes (e.g. going from local to production), you change ONE file.

```
nano ~/shopnest/frontend/src/services/api.js
```

```
import axios from 'axios';

// Create an axios instance with the base URL from the .env file
const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  headers: { 'Content-Type': 'application/json' },
});

// Request interceptor – attach the JWT token to every request automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor – handle 401 errors globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

// — Auth —
export const registerUser = (data) => api.post('/auth/register', data);
export const loginUser = (data) => api.post('/auth/login', data);

// — Products —
export const getProducts = (params) => api.get('/products', { params });
export const getProductById = (id) => api.get(`/products/${id}`);

// — Cart —
export const getCart = () => api.get('/cart');
export const addToCart = (data) => api.post('/cart', data);
export const removeFromCart = (productId) => api.delete(`/cart/${productId}`);

// — Orders —
export const placeOrder = () => api.post('/orders');
export const getOrders = () => api.get('/orders');
```

5.4 — Create the Main Pages

STEP 27 Create src/pages/ProductsPage.jsx

```
nano ~/shopnest/frontend/src/pages/ProductsPage.jsx
```

```
import React, { useState, useEffect } from 'react';
import { getProducts, addToCart } from '../services/api';

function ProductsPage() {
  const [products, setProducts] = useState([]);
```

```

const [loading, setLoading] = useState(true);
const [message, setMessage] = useState('');

useEffect(() => {
  getProducts()
    .then(res => setProducts(res.data.products))
    .catch(err => console.error('Failed to load products', err))
    .finally(() => setLoading(false));
}, []);

const handleAddToCart = async (productId) => {
  try {
    await addToCart({ productId, quantity: 1 });
    setMessage('Added to cart!');
    setTimeout(() => setMessage(''), 2000);
  } catch (err) {
    setMessage(err.response?.data?.error || 'Please log in to add items');
  }
};

if (loading) return <p>Loading products...</p>;

return (
  <div style={{ padding: '20px' }}>
    <h1>ShopNest Products</h1>
    {message && <p style={{ color: 'green', fontWeight: 'bold' }}>{message}</p>}
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3, 1fr)', gap: '20px' }}>
      {products.map(product => (
        <div key={product.id} style={{ border: '1px solid #ddd', padding: '15px', borderRadius: '8px' }}>
          <h3>{product.name}</h3>
          <p>{product.description}</p>
          <p style={{ fontSize: '1.3rem', fontWeight: 'bold', color: '#0E7C7B' }}>
            ${product.price}
          </p>
          <p style={{ color: product.stock > 0 ? 'green' : 'red' }}>
            {product.stock > 0 ? `${product.stock} in stock` : 'Out of stock'}
          </p>
          <button
            onClick={() => handleAddToCart(product.id)}
            disabled={product.stock === 0}
            style={{ background: '#0E7C7B', color: 'white', border: 'none', padding: '8px 16px', borderRadius: '4px', cursor: 'pointer' }}
          >
            Add to Cart
          </button>
        </div>
      ))}
    </div>
  </div>
);
}

export default ProductsPage;

```

STEP 28 Create src/pages/LoginPage.jsx

```
nano ~/shopnest/frontend/src/pages/LoginPage.jsx
```

```
import React, { useState } from 'react';
import { loginUser } from '../services/api';

function LoginPage() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError('');
    try {
      const res = await loginUser(form);
      localStorage.setItem('token', res.data.token);
      window.location.href = '/products';
    } catch (err) {
      setError(err.response?.data?.error || 'Login failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ maxWidth: '400px', margin: '80px auto', padding: '30px',
border: '1px solid #ddd', borderRadius: '8px' }}>
      <h2>Login to ShopNest</h2>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '15px' }}>
          <label>Email</label>
          <input type='email' value={form.email}
            onChange={e => setForm({...form, email: e.target.value})}
            style={{ display: 'block', width: '100%', padding: '8px', marginTop:
'4px' }}
            required />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label>Password</label>
          <input type='password' value={form.password}
            onChange={e => setForm({...form, password: e.target.value})}
            style={{ display: 'block', width: '100%', padding: '8px', marginTop:
'4px' }}
            required />
        </div>
        <button type='submit' disabled={loading}
          style={{ background: '#1B2A4A', color: 'white', border: 'none',
padding: '10px 20px', borderRadius: '4px', cursor: 'pointer', width: '100%' }}>
          {loading ? 'Logging in...' : 'Login'}
        </button>
      </form>
      <p style={{ marginTop: '15px', textAlign: 'center' }}>
        No account? <a href='/register'>Register here</a>
      </p>
    </div>
  );
}

export default LoginPage;
```

STEP 29 Update src/App.js with routing

```
nano ~/shopnest/frontend/src/App.js
```

Delete all existing content and replace with:

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
import ProductsPage from './pages/ProductsPage';
import LoginPage from './pages/LoginPage';

function App() {
  const token = localStorage.getItem('token');

  const handleLogout = () => {
    localStorage.removeItem('token');
    window.location.href = '/login';
  };

  return (
    <Router>
      <nav style={{ background: '#1B2A4A', padding: '15px 30px', display: 'flex',
        justifyContent: 'space-between', alignItems: 'center' }}>
        <Link to='/products' style={{ color: '#0E7C7B', fontSize: '1.4rem',
          fontWeight: 'bold', textDecoration: 'none' }}>
          ShopNest
        </Link>
        <div>
          {token
            ? <button onClick={handleLogout} style={{ color: 'white', background:
              'transparent', border: '1px solid white', padding: '6px 14px', cursor: 'pointer',
              borderRadius: '4px' }}>Logout</button>
            : <Link to='/login' style={{ color: 'white', textDecoration: 'none'
              }}>Login</Link>
          }
        </div>
      </nav>
      <Routes>
        <Route path="/" element={<ProductsPage />} />
        <Route path="/products" element={<ProductsPage />} />
        <Route path="/login" element={<LoginPage />} />
      </Routes>
    </Router>
  );
}

export default App;
```

STEP 30 Start the React development server

```
cd ~/shopnest/frontend
npm start

# React starts on port 3000
# Your browser should open automatically at http://localhost:3000
```

OK Success Check

1. Open `http://localhost:3000` in your browser
2. You should see the ShopNest navbar and a list of 6 products
3. If you see products — the frontend is calling the backend API successfully
4. If you see 'Loading...' or an error — check that the backend is still running on port 5000

STEP 6 — TEST ALL API ENDPOINTS WITH CURL

Now we test every API endpoint systematically. Open a NEW terminal for these tests. Keep the backend and frontend terminals running.

i Why Test With curl?

curl is the universal HTTP testing tool — it works identically in your terminal, inside a CI/CD pipeline, and in any cloud environment.

Learning to test with curl is a core DevOps skill used for health checks, pipeline validation, and debugging production issues.

6.1 — Test the Health Check

STEP 31 Test GET /health

```
curl -s http://localhost:5000/health | python3 -m json.tool
```

```
# Expected output:
```

```
# {  
#   "status": "healthy",  
#   "timestamp": "2024-01-15T10:30:00.000Z",  
#   "uptime": 45.2,  
#   "version": "1.0.0"  
# }
```

OK What this tells you

If this returns 200 with 'healthy' — the API is running.

This is the same endpoint the AWS ALB and Kubernetes liveness probe will call.

6.2 — Test Auth Endpoints

STEP 32 Test POST /api/auth/register

```
curl -s -X POST http://localhost:5000/api/auth/register \  
-H 'Content-Type: application/json' \  
-d '{  
  "email": "test@shopnest.com",  
  "password": "Test@12345",  
  "first_name": "Test",  
  "last_name": "User"  
}' | python3 -m json.tool
```

```
# Expected output:
```

```
# {  
#   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
#      "user": {
#          "id": 1,
#          "email": "test@shopnest.com",
#          "first_name": "Test",
#          "last_name": "User",
#          "role": "customer"
#      }
# }

# IMPORTANT: Copy the token value – you need it for the next tests
```

STEP 33 Test POST /api/auth/login and save the token

```
# Login and save the token to a shell variable
TOKEN=$(curl -s -X POST http://localhost:5000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email": "test@shopnest.com", "password": "Test@12345"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")

# Verify the token was saved
echo $TOKEN
# Should print a long JWT string starting with eyJ...
```

6.3 — Test Product Endpoints

STEP 34 Test GET /api/products

```
# Get all products
curl -s http://localhost:5000/api/products | python3 -m json.tool

# Get products filtered by category
curl -s 'http://localhost:5000/api/products?category=Sports' | python3 -m json.tool

# Get a single product by ID
curl -s http://localhost:5000/api/products/1 | python3 -m json.tool
```

6.4 — Test Cart Endpoints

STEP 35 Test cart add, get, and remove

```
# Add product 1 to cart (requires TOKEN from Step 33)
curl -s -X POST http://localhost:5000/api/cart \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"productId": 1, "quantity": 2}' | python3 -m json.tool

# Add product 2 to cart
curl -s -X POST http://localhost:5000/api/cart \
  -H 'Content-Type: application/json' \
```

```
-H "Authorization: Bearer $TOKEN" \  
-d '{"productId": 2, "quantity": 1}' | python3 -m json.tool  
  
# Get the cart  
curl -s http://localhost:5000/api/cart \  
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool  
  
# Remove product 2 from cart  
curl -s -X DELETE http://localhost:5000/api/cart/2 \  
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

6.5 — Test Order Endpoint

STEP 36 Test POST /api/orders — place an order

```
# Place an order (converts cart to order and clears the cart)  
curl -s -X POST http://localhost:5000/api/orders \  
-H 'Content-Type: application/json' \  
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool  
  
# Expected output:  
# {  
#   "order": {  
#     "id": 1,  
#     "user_id": 1,  
#     "status": "pending",  
#     "total_amount": "179.98",  
#     ...  
#   },  
#   "message": "Order placed successfully"  
# }  
  
# Get all orders for this user  
curl -s http://localhost:5000/api/orders \  
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

STEP 7 — PHASE 2 VALIDATION & TROUBLESHOOTING

7.1 — Full Validation Checklist

Run through every item. All must pass before Phase 2 is complete.

OK Phase 2 Complete When ALL of These Pass

DATABASE

- ☐ PostgreSQL is running: `sudo systemctl status postgresql`
- ☐ shopnest_db exists: `psql -U shopnest_user -h localhost -d shopnest_db -W -c '\dt'`
- ☐ All 4 tables exist: users, products, orders, order_items
- ☐ 6 seed products are in the database

REDIS

- ☐ Redis is running: `redis-cli ping` returns PONG

BACKEND API

- ☐ Server starts without errors: `npm run dev` in backend/
- ☐ Health check returns 'healthy': `curl http://localhost:5000/health`
- ☐ Register endpoint works (Step 32)
- ☐ Login endpoint works and returns a JWT token (Step 33)
- ☐ Products endpoint returns 6 products (Step 34)
- ☐ Cart add/get/remove all work (Step 35)
- ☐ Order placement works (Step 36)

FRONTEND

- ☐ React app starts: `npm start` in frontend/
- ☐ Products page loads at `http://localhost:3000` showing all 6 products
- ☐ Login page works at `http://localhost:3000/login`
- ☐ No CORS errors in browser developer tools (F12 -> Console tab)

7.2 — Common Errors and Fixes

X Error: ECONNREFUSED 127.0.0.1:5432

PostgreSQL is not running.

Fix: `sudo systemctl start postgresql`

Then: `sudo systemctl status postgresql` to confirm it started.

X Error: ECONNREFUSED 127.0.0.1:6379

Redis is not running.

Fix: `sudo systemctl start redis-server`

Then: `redis-cli ping` — should return PONG.

X Error: role 'shopnest_user' does not exist

You skipped Step 2 or it failed partway through.

Fix: Run Step 2 again from the beginning.

Check: `sudo -u postgres psql -c '\du'` should list `shopnest_user`.

X CORS Error in browser: Access-Control-Allow-Origin missing

The frontend URL in your backend `.env` does not match what the browser is using.

Check: `cat ~/shopnest/backend/.env` — `FRONTEND_URL` must be `http://localhost:3000`

Fix: Update `FRONTEND_URL` and restart the backend.

X 401 Unauthorized on cart/order endpoints

You are not passing the JWT token in the request header.

Check that `$TOKEN` is set: `echo $TOKEN` — must print a long string.

If empty, run Step 33 again to re-login and capture the token.

! React shows 'Loading...' and never loads products

The React app cannot reach the backend API.

Check 1: Is the backend running? `curl http://localhost:5000/health`

Check 2: Is `REACT_APP_API_URL` set correctly in `frontend/.env`?

Check 3: After editing `.env`, stop React (Ctrl+C) and run `npm start` again — React does NOT hot-reload `.env` changes.

7.3 — Understanding the Data Flow You Just Built

Now that everything is working, trace the complete data flow one more time — this time it is real code, not just a diagram.

Step	What happens	Code location
1	Browser opens <code>http://localhost:3000</code>	React Dev Server
2	<code>App.js</code> renders <code>ProductsPage</code> component	<code>frontend/src/App.js</code>
3	<code>useEffect</code> calls <code>getProducts()</code> on page load	<code>frontend/src/pages/ProductsPage.jsx</code>

4	api.js sends GET http://localhost:5000/api/products	frontend/src/services/api.js
5	Express receives the request	backend/src/index.js
6	Routes it to productRoutes.js -> getAllProducts()	backend/src/routes/productRoutes.js
7	Controller queries PostgreSQL: SELECT * FROM products	backend/src/controllers/productController.js
8	PostgreSQL returns 6 rows	PostgreSQL at localhost:5432
9	Controller sends JSON response back to React	backend/src/controllers/productController.js
10	React renders product cards on the page	frontend/src/pages/ProductsPage.jsx

WHAT COMES NEXT — PHASE 3 PREVIEW

i Phase 3 — Git & Version Control

In Phase 3 you will take everything you built in Phase 2 and set it up for team collaboration.

You will:

- Create a GitHub repository and push the project
- Set up a proper branching strategy (main / develop / feature branches)
- Write meaningful commit messages following Conventional Commits standard
- Create a Pull Request workflow with branch protection rules
- Set up CODEOWNERS to require reviews on sensitive files
- Create a complete .gitignore that handles all edge cases
- Understand why this exact branching strategy connects to Jenkins in Phase 5

Phase 3 takes 30-45 minutes and sets up the version control foundation that CI/CD builds on.

Phase 2 Quick Reference Card

Service	URL	How to start	How to stop
PostgreSQL	localhost:5432	sudo systemctl start postgresql	sudo systemctl stop postgresql
Redis	localhost:6379	sudo systemctl start redis-server	sudo systemctl stop redis-server
Backend API	localhost:5000	cd backend && npm run dev	Ctrl+C in backend terminal
Frontend React	localhost:3000	cd frontend && npm start	Ctrl+C in frontend terminal

ShopNest DevOps Project — Phase 2 | Reply 'Phase 3' to continue